COMP353 Assignment 2 Joshua Visario

Question 1

Schemas to guide my ER diagram

Patients (SIN, name, medicareNumber, insurance, policyNumberInsurance, homeAddress, emailAddress, phonenumber, DOB)

SIN	name	medicareCardNumber	insurance	policyNumber	homeAddress	emailAddress	phoneNumber	dateOfBirth

Doctor (SIN, name, address, emailAddress, phoneNumber, DOB, specialty, yearsOfExperience)

SIN	name	homeAddress	emailAddress	phoneNumber	dateOfBirth	specialty	yearsOfExperience

Pharmacists (SIN, name, address, emailAddress, phoneNumber, DOB, yearsOfExperience)

SIN	name	homeAddress	emailAddress	phoneNumber	dateOfBirth	yearsOfExperience

Pharmaceutical Company (name, websiteAddress, emailAddress, pharmAddress, phoneNumber)

pharmCompanyName	websiteAdress	emailAddress	companyAddress	phoneNumber

Drug (Weak Entity) (tradeName, pharmaceuticalCompany, formula)

tradeName (partial key) (Unique within its pharmaceutical company)	formula

Pharmacy(pharmacyID, pharmacyName, pharmAddress, websiteAddress, emailAdress, phoneNumber)

pharmacyID	pharmacyName	pharmAddress	websiteAddress	emailAddress	phoneNumber

pharmacyID added because not all pharmacy names are unique

Additional Entities

Contract (Separate entity) (initially a relationship) (contractID, startDate, endDate, contractText)

contractID?	startDate	endDate	contractText

PrescriptionPurchase Entity

purchaseID

Prescription (Separate Entity(initially a relationship) (prescriptionID, prescriptionDate, quantity)

prescriptionID	prescriptionDate	quantity

Relationships:

1) **hasPrimaryPhysician** Relationship

Every patient has a primary physician. Every doctor has at least one patient, and possibly several. PATIENT ------ DOCTOR

Cardinality: From Patient to Doctor: Many to One (M:1)

```
[Patient 1]---------------|
[Patient 2]---------------| <hasPrimaryPhysician>------------->[Doctor]
[Patient 3]---------------|
```

2) **Produces** Relationship (Weak Relationship)

- One pharmaceuticalCompany produces many drugs
- Each drug belongs to exactly one company
- Drug is weak

Cardinality: From Drug to Pharmaceutical Company: Many to One (M:1)

```
                                                    |--------------[Drug]
[Pharmaceutical Company]<---------| <Produces>--------------[Drug]
                                                    |--------------[Drug]
```

3) **Sells** Relationship

- Each pharmacy sells several drugs and has a price for each. A drug could be sold at several pharmacies, and the price could vary from one pharmacy to another

PHARMACY ----- SELLS ----DRUG

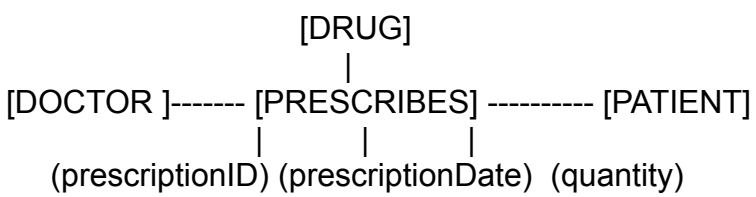
Cardinality: Many to Many (M:N)

```
[PHARMACY 1]---------------|           |----------[DRUG1]
[PHARMACY 2]---------------<SELLS> ---------[DRUG2]
[PHARMACY 3]---------------|     |     |----------[DRUG3]
                                 |
                              (PRICE)
```

4) **Prescribes** Relationship (Initially) → Turned into Prescription Entity (Associative)

- Doctors prescribe drugs for patients. A doctor could prescribe one or more drugs for several patients, and a patient could obtain prescriptions from several doctors.
- Each prescription has a date and a quantity associated with it. You can assume that if a doctor prescribes the same drug for the same patient more than once, only the last prescription needs to be stored.

Ternary Relationship



5) Contract Relationship (Initially) Turned into Contract Entity (Associative)
Pharmaceutical companies have long-term contracts with pharmacies. A pharmaceutical company can contract with several pharmacies, and a pharmacy can contract with several pharmaceutical companies. For each contract, you have to store a start date, and end date, and the text of the contract.
COMPANY-------- PHARMACY

Cardinality: Pharmacy and PharmaceuticalCompany: M:N Through the contract entity



If multiple contracts must coexists, you may need a separate Contract entity with a way to identify each contract with its own identifier

6) Employs Relationship (created)

- One pharmacy employs many pharmacist
- Each pharmacist works for one pharmacy
-

Pharmacy -- Employs --- Pharmacist

Each pharmacy must have exactly one main pharmacist
Cardinality: 1:N (Still despite only one main pharmacist as a constaint

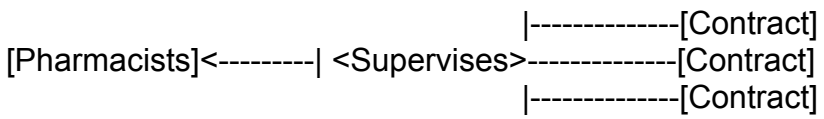
[Pharmacy] <----- <Employs> ------- [Pharmacist]

Assumption: Pharmacist works at only one pharmacy

7) hasMainPharmacist Relationship(created)

[PHARMACY]----------| <hasMainPharmacist> ------>Pharmacist

8) Supervises Relationship



At all times, there must be a supervisor for each contract

9) ApprovedBy Relationship

Cardinality: N:1

[PrescriptionPurchase]----------|
[PrescriptionPurchase]----------| <Approves>---------->[Pharmacist]
[PrescriptionPurchase]----------|

Additional Relationships

10) **ForPatient** Relationship

Every prescription is associated with exactly one patient. A patient may have several prescriptions
Cardinality: N:1, From Prescription to Patient

Prescription → Patient

[Prescription]----------|
[Prescription]----------| <ForPatient>---------->[Patient]
[Prescription]----------|

11) **IssuedBy** Relationship

Every prescription is issued by exactly one doctor. A doctor may issues several prescriptions

Cardinality: N:1, From Prescription to Doctor

Prescription → Doctor

[Prescription]----------|
[Prescription]----------| <IssuedBy>---------->[Doctor]
[Prescription]----------|

12) **ForDrug** Relationship
Each prescription is associated with exactly one drug. A drug may appear in prescriptions issued to several patients.
Cardinality: N:1, From Prescription to Drug

[Prescription]----------|
[Prescription]----------| <ForDrug>---------->[Drug]
[Prescription]----------|

13) **Fulfills** Relationship

Each prescription purchase is associated with exactly one prescription. A prescription may be associated with multiple purchase records.

Cardinality: N:1, From PrescriptionPurchase to Prescription

PrescriptionPurchase → Prescription

[PrescriptionPurchase]----------|
[PrescriptionPurchase]----------| <Fulfills>---------->[Prescription]
[PrescriptionPurchase]----------|

14) **purchasedAt** Relationship

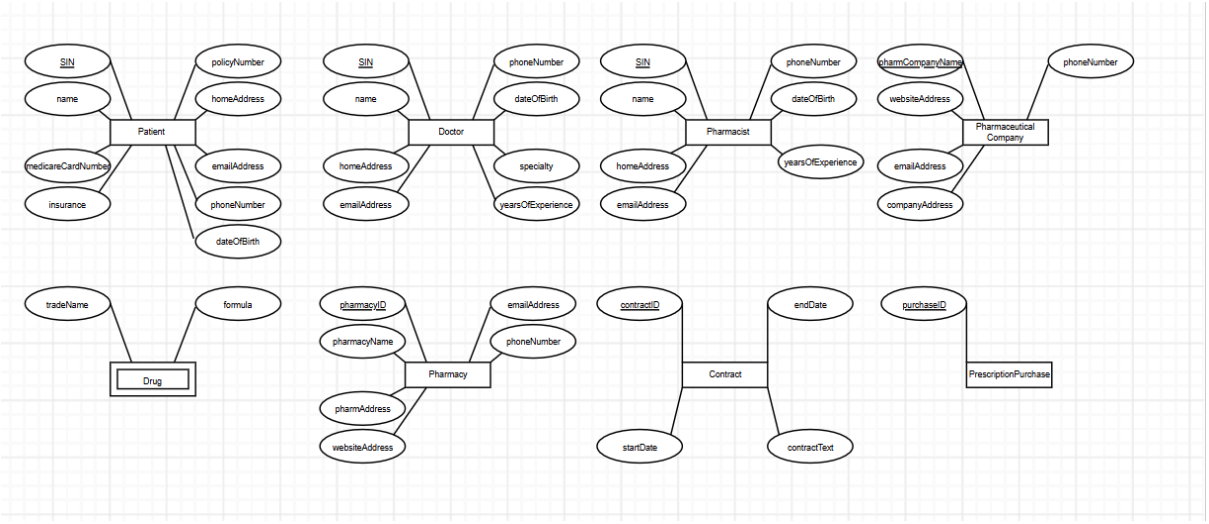
Each prescription purchase occurs at exactly one pharmacy. A pharmacy can process many prescription purchases.

Cardinality: N:1, From PrescriptionPurchase to Pharmacy

PrescriptionPurchase → Pharmacy

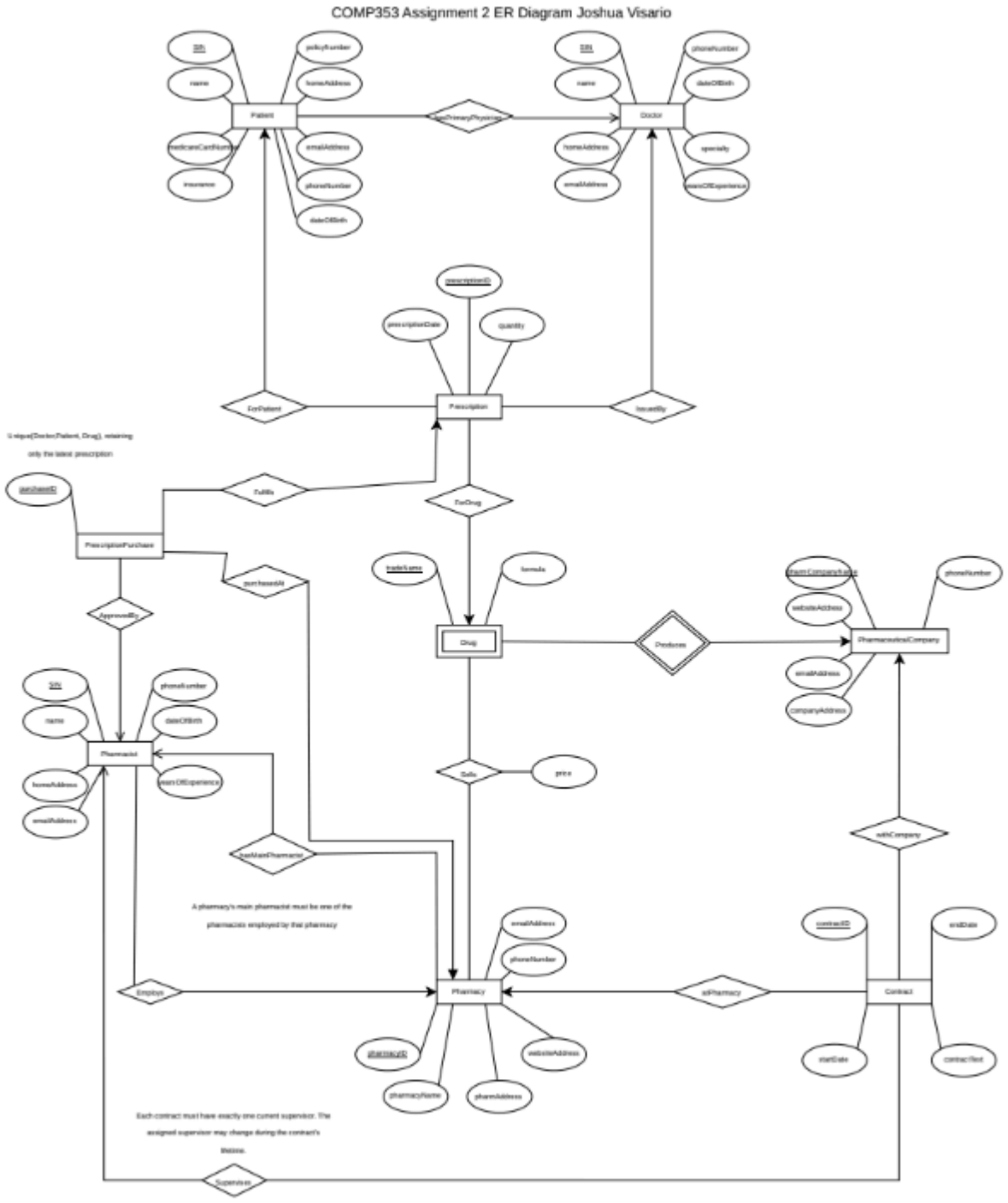
[PrescriptionPurchase]----------|
[PrescriptionPurchase]----------| <purchasedAt>---------->[Pharmacy]
[PrescriptionPurchase]----------|

Initial ER Diagram for each Entity



Official ER Diagram

- Prescription became an entity
- Contract became an Entity



Constraints Not Fully Captured by the ER Diagram

1. **Prescription uniqueness and recency:** For each combination of Doctor, Patient, and Drug, only one prescription is stored. If the same doctor prescribes the same drug to the same patient again, the existing prescription record is updated to retain the latest prescription date and quantity.
2. **Main pharmacist employment:** Every pharmacy must have exactly one main pharmacist, and that pharmacist must also be employed at the same pharmacy. This consistency requirement across the Employs and HasMainPharmacist relationships is not enforced by ordinary cardinality arrows alone.
3. **Contract supervision:** Every contract must have exactly one current supervisor. The assigned supervisor may change over the contract's lifetime. This design stores only the current supervisor and does not preserve previous supervisor assignments.
4. **Minimum relationship requirements:** Every doctor must have at least one patient, and every pharmacy must sell several drugs. Ordinary cardinality arrows constrain maximum multiplicity but do not express these minimum-count requirements.
5. **Prescription purchase approval:** Every prescription purchase must be verified and approved by exactly one pharmacist. Ordinary multiplicity notation alone does not guarantee that an approving pharmacist exists.
6. **Drug dependency:** Drug is a weak entity identified by its trade name and the pharmaceutical company that produces it. In the SQL implementation, a pharmaceutical company cannot be deleted while its associated drugs still reference it, unless those drug records are deleted first.

3. [15 Points] Convert your ER diagram into a relational database scheme. Make refinements to your scheme if possible. Identify the primary keys and the foreign keys in the relational schemes, and hence note the referential integrity constraints in the scheme.

ER Diagram (Conceptual Design) to Relational Database schema (Logical Design)

PK = Primary Key, uniquely identifying each row

FK = Foreign Key, referencing a key in another table

Composite PK = Primary Key consisting of multiple attributes

NOT NULL = Required Value

Patient (patientSIN (PK), name, medicareCardNumber, insurance, policynumber, homeAddress, emailAddress, phoneNumber, dateOfBirth, primaryDoctorSIN(FK)

patientSIN (PK)	name	medicareCardNumber	insurance	policyNumber	homeAddress	emailAddress	phoneNumber	dateOfBirth	primaryDoctorSIN (FK)

Doctor (doctorSIN(PK), name, homeAddress, emailAddress, phoneNumber, dateOfBirth, specialty, yearsOfExperience)

doctorSIN(PK)	name	homeAddress	emailAddress	phoneNumber	dateOfBirth	specialty	yearsOfExperience

Every doctor must have at least one patient (`Patient.primaryDoctorSIN NOT NULL` ensures that every patient is assigned a primary physician.

Pharmacist (pharmacistSIN(PK), name, homeAddress, emailAddress, phoneNumber, dateOfBirth, yearsOfExperience, pharmaacyID(FK))

pharmacistSIN(PK)	name	homeAddress	emailAddress	phoneNumber	dateOfBirth	yearsOfExperience	pharmacyID(FK)

UNIQUE(Pharmacist.pharmacistSIN, Pharmacist.pharmacyID)

Pharmaceutical Company (pharmCompanyName(PK), websiteAddress, emailAddress, companyAddress, phonenumber)

pharmCompanyName(PK)	websiteAdress	emailAddress	companyAddress	phoneNumber

Drug (Weak Entity) (tradeName (PK component), pharmCompanyName (PK component, FK), formula) . Composite key PK(tradeName, pharmCompanyName)

tradeName (PK)	pharmCompanyName (PK+FK)	formula

Pharmacy (pharmacyID (PK), pharmacyName, pharmAddress, websiteAddress, emailAddress, phoneNumber, mainPharmacistSIN(FK))

pharmacyID (PK)	pharmacyName	pharmAddress	websiteAddress	emailAddress	phoneNumber	mainPharmacistSIN(FK)

pharmacyID because not all pharm names are unique. Every pharmacy has a main pharmacist (Pharmacy.mainPharmacistSIN NOT NULL) Every pharmacy must sell several drugs. mainpharmacist must work at the pharmacy they manage
UNIQUE(Pharmacist.pharmacistSIN, Pharmacist.pharmacyID)
FOREIGN KEY (mainPharmacistSIN, pharmacyID)
REFERENCES Pharmacist(pharmacistSIN, pharmacyID)

New Entities in place

Prescription (prescriptionID (PK), prescriptionDate,quantity, tradeName(FK), pharmCompanyName (FK), patientSIN (FK), doctorSIN (FK) /UNIQUE (doctorSIN, patientSIN, tradeName, pharmCompanyName) NOT NULL

prescriptionID (PK)	prescriptionDate	quantity	tradeName(FK)	pharmCompanyName (FK)	patientSIN (FK)	doctorSIN (FK)

FOREIGN KEY (tradeName, pharmCompanyName)
REFERENCES Drug(tradeName, pharmCompanyName)

Proposed Entities

Contract (Separate entity) (contractID (PK), startDate, endDate, contractText, pharCompanyName(FK), pharmacyID(FK), supervisorSIN(FK)

contractID (PK)	startDate	endDate	contractText	pharCompanyName(FK)	pharmacyID(FK)	supervisorSIN(FK)

The current contract supervisor must exist at all times Contract.supervisorSIN NOT NULL

PrescriptionPurchase Entity (purchaseID (PK), prescriptionID (FK),pharmacyID(FK), approvedbySIN(FK))

purchaseID (PK)	prescriptionID (FK)	pharmacyID(FK)	approvedbySIN(FK)

Only the latest prescription is stored for each doctor-patient-drug combination. The unique constraint prevents duplicates. Approving pharmacist must work at the pharmacy where the purchase occurs. If a prescription purchase must involved a drug sold by that pharmacy, that consistency also needs ot be enforced PrescriptionPurchase.approvedBySIN NOT NULL `prescriptionID NOT NULL pharmacyID NOT NULL`

Sells Relationship (Only one with attributes) (pharmacyID (PK)(FK), tradeName (PK)(FK), pharmCompanyName (PK)(FK), price)

pharmacyID (PK)(FK)	tradeName (PK)(FK)	pharmCompanyName (PK)(FK)	price

FOREIGN KEY (tradeName, pharmCompanyName) REFERENCES Drug(tradeName, pharmCompanyName)
FOREIGN KEY (pharmacyID) REFERENCES Pharmacy(pharmacyID)

4. [5 Points] How would your design change if each drug is required to be sold at a fixed price by all pharmacies?

Before: Different prices at different pharmacies
I had: Pharmacy - Sells - Drug (price has the attribute of Sells)
Since the price depends on both the pharmacy and the drug, it belongs to the Sells Relationship

Pharmacy	**Drug**	**Price**
Jean Coutu	Aspirin	$10.00
Pharmaprix	Aspirin	$12.00
Walmart	Aspirin	$9.00

Now: Every pharmacy must sell a given drug at the same price

Pharmacy	**Drug**	**Price**
Jean Coutu	Aspirin	$10.00
Pharmaprix	Aspirin	$10.00
Walmart	Aspirin	$10.00

The price no longer depend son which pharmacy sells the drug. It depends on only the drug itself. Thus, instead of being an attribute of the Sells Relationship, it will be an attribute of the Drug Entity.

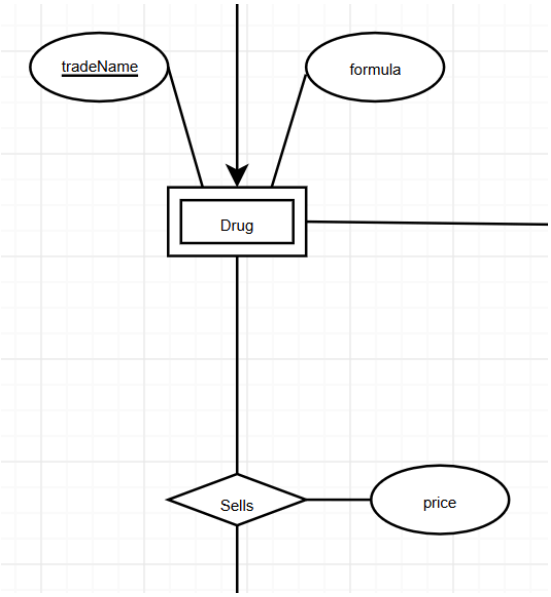
Design Change:

- Remove the price attribute from the *Sells* relationship
- Add *price* as an attribute of the *Drug* entity
- Keep the *Sells* relationship between Pharmacy and Drug because a pharmacy can still sell multiple drugs and a drug can still be sold by multiple pharmacies

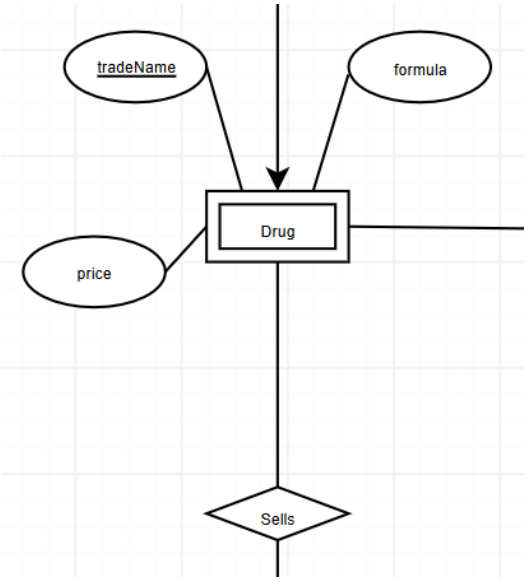
Relational Schema Modification:

- Before:
  - Drug(tradName, pharmCompanyName, formula)
  - Sells(pharmacyID, tradeName, pharmCompanyName, price)
- After:
  - Drug(tradename, pharmCompanyName, formula, price)
  - Sells(pharmacyID, tradeName, pharmCompanyName)

Before:



After:



Part II

Part II (25 points):

Express the following queries in **SQL:**

a) [10 Points] For the above relational database scheme in part I, write SQL "CREATE TABLE" statements using appropriate data types for the various attributes. Include specifications of keys, foreign keys and NOT NULL constraints..

```
-- COMP 353 -- Pharmacy Database
-- PART II
-- a) TABLES Creation

DROP DATABASE IF EXISTS pharmacy_database;
CREATE DATABASE IF NOT EXISTS pharmacy_database;
USE pharmacy_database;

-- DOCTOR
CREATE TABLE Doctor (
    doctorSIN CHAR(9) PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    homeAddress VARCHAR(255),
    emailAddress VARCHAR(255),
    phoneNumber VARCHAR(255),
    dateOfBirth DATE,
    specialty VARCHAR(100),
    yearsOfExperience INT
);

-- PATIENT
CREATE TABLE Patient (
    patientSIN CHAR(9) PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    medicareCardNumber VARCHAR(20),
    insurance VARCHAR(100),
    policyNumber VARCHAR(50),
    homeAddress VARCHAR(255),
    emailAddress VARCHAR(255),
    phoneNumber VARCHAR(255),
    dateOfBirth DATE,
    primaryDoctorSIN CHAR(9) NOT NULL,

    -- Foreign key constraint to Primary Doctor
```

```
    CONSTRAINT fk_patient_doctor
        FOREIGN KEY (primaryDoctorSIN)
        REFERENCES Doctor(doctorSIN)
);

-- PHARMACEUTICAL COMPANY
CREATE TABLE PharmaceuticalCompany(
    pharmCompanyName VARCHAR(100) PRIMARY KEY,
    websiteAddress VARCHAR(255),
    emailAddress VARCHAR(255),
    companyAddress VARCHAR(255),
    phoneNumber VARCHAR(20)
);


-- PHARMACY
CREATE TABLE Pharmacy (
    pharmacyID CHAR(9) PRIMARY KEY,
    pharmacyName VARCHAR(100) NOT NULL,
    pharmAddress VARCHAR(255),
    websiteAddress VARCHAR(255),
    emailAddress VARCHAR(255),
    phoneNumber VARCHAR(20),
    mainPharmacistSIN CHAR(9) NULL -- Temporarily

    -- Main pharmacist FK will be added later
);

-- PHARMACIST
CREATE TABLE Pharmacist (
    pharmacistSIN CHAR(9) PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    homeAddress VARCHAR(255),
    emailAddress VARCHAR(255),
    phoneNumber VARCHAR(20),
    dateOfBirth DATE,
    yearsOfExperience INT,
    pharmacyID CHAR(9) NOT NULL,

    -- Unique constraint that a pharmacists works only at one pharmacy
    -- COMPOSITE KEY
    CONSTRAINT uq_pharmacist_workplace
        UNIQUE (pharmacistSIN, pharmacyID),
```

```
    -- Foreign key constraint to pharmacyID
    CONSTRAINT fk_pharmacist_pharmacy
        FOREIGN KEY (pharmacyID)
        REFERENCES Pharmacy(pharmacyID)
);

-- Adding MAIN PHARMACIST FOREIGN KEY
-- Create Pharmacy, Create Pharmacist, Add main pharmacist foreign key
afterward
-- ALTER TABLE Pharmacy
-- ADD CONSTRAINT fk_pharmacy_main_pharmacist
--    FOREIGN KEY (mainPharmacistSIN, pharmacyID)
--    REFERENCES Pharmacist(pharmacistSIN, pharmacyID);


-- DRUG (WEAK ENTITY)
CREATE TABLE Drug (
    tradeName VARCHAR(100) NOT NULL,
    pharmCompanyName VARCHAR(100) NOT NULL,
    formula VARCHAR(255),

    -- Since DRUG is a weak entity, it has a COMPOSITE KEY(it's
discriminator and the key of the strong entity
    PRIMARY KEY (tradeName, pharmCompanyName),

    -- Foreign key constraint to pharCompanyName
    CONSTRAINT fk_drug_company
        FOREIGN KEY (pharmCompanyName)
        REFERENCES PharmaceuticalCompany(pharmCompanyName)
);


-- PRESCRIPTION
CREATE TABLE Prescription (
    prescriptionID INT PRIMARY KEY,
    prescriptionDate DATE NOT NULL,
    quantity INT NOT NULL,
    tradeName VARCHAR(100) NOT NULL,
    pharmCompanyName VARCHAR(100) NOT NULL,
    patientSIN CHAR(9) NOT NULL,
    doctorSIN CHAR(9) NOT NULL,
```

```
    -- Unique Constraint Prescrption relating to the latest
prescription
    CONSTRAINT uq_prescription_combination
        UNIQUE (
            doctorSIN,
            patientSIN,
            tradeName,
            pharmCompanyName
        ),

    -- Foreign key constraint to DoctorSIN
    CONSTRAINT fk_prescription_doctor
        FOREIGN KEY (doctorSIN)
        REFERENCES Doctor(doctorSIN),

    -- Foreign key contraint to PatientSIN
    CONSTRAINT fk_prescription_patient
        FOREIGN KEY (patientSIN)
        REFERENCES Patient(patientSIN),

    -- Foreign Key constraint to the weak Drug Entity
    CONSTRAINT fk_prescription_drug
        FOREIGN KEY (tradeName, pharmCompanyName)
        REFERENCES Drug(tradeName, pharmCompanyName)
);

-- CONTRACT
CREATE TABLE Contract (
    contractID INT PRIMARY KEY,
    startDate DATE NOT NULL,
    endDate DATE NOT NULL,
    contractText TEXT NOT NULL,
    pharmCompanyName VARCHAR(100) NOT NULL,
    pharmacyID CHAR(9) NOT NULL,
    supervisorSIN CHAR(9) NOT NULL,

    -- Foreign key constraint to pharmCompanyName
    CONSTRAINT fk_contract_company
        FOREIGN KEY (pharmCompanyName)
        REFERENCES PharmaceuticalCompany(pharmCompanyName),

    -- Foreign Key constraint to pharmacyID
    CONSTRAINT fk_contract_pharmacy
```

```
        FOREIGN KEY (pharmacyID)
        REFERENCES Pharmacy(pharmacyID),

    -- Foreign Key constraint to pharmacistSIN
    CONSTRAINT fk_contract_supervisor
        FOREIGN KEY (supervisorSIN)
        REFERENCES Pharmacist(pharmacistSIN)
);

-- PRESCRIPTION PURCHASE
CREATE TABLE PrescriptionPurchase (
    purchaseID INT PRIMARY KEY,
    prescriptionID INT NOT NULL,
    pharmacyID CHAR(9) NOT NULL,
    approvedBySIN CHAR(9) NOT NULL,

    -- Foreign key to prescriptionID
    CONSTRAINT fk_purchase_prescription
        FOREIGN KEY (prescriptionID)
        REFERENCES Prescription(prescriptionID),

    -- Foreign key to pharmacyID
    CONSTRAINT fk_purchase_pharmacy
        FOREIGN KEY (pharmacyID)
        REFERENCES Pharmacy(pharmacyID),

    -- Approving pharmacist must work
    -- at the pharmacy where the purchase occurs
    CONSTRAINT fk_purchase_approver_workplace
        FOREIGN KEY (approvedBySIN, pharmacyID)
        REFERENCES Pharmacist(pharmacistSIN, pharmacyID)
);


-- SELLS (M:N Relationship)

CREATE TABLE Sells (
    pharmacyID CHAR(9) NOT NULL,
    tradeName VARCHAR(100) NOT NULL,
    pharmCompanyName VARCHAR(100) NOT NULL,
    price DECIMAL(10,2) NOT NULL,

    -- Since Drug Entity is a weak entity
```

```
    PRIMARY KEY (
        pharmacyID,
        tradeName,
        pharmCompanyName
    ),

    -- Foreign key to pharmacyID
    CONSTRAINT fk_sells_pharmacy
        FOREIGN KEY (pharmacyID)
        REFERENCES Pharmacy(pharmacyID),

    -- Foreign key to Drug Entity
    CONSTRAINT fk_sells_drug
        FOREIGN KEY (tradeName, pharmCompanyName)
        REFERENCES Drug(tradeName, pharmCompanyName)
);


-- Testings
SHOW TABLES;
DESCRIBE Doctor;
```

b) [5 Points] For each table you created in part (a), provide one INSERT statement that will populate the table with data.

```
--
------------------------------------------------------------------------
------------
-- b) Providing one INSERT statement that will populate the table with
data

-- DOCTOR
INSERT INTO Doctor
(doctorSIN, name, homeAddress, emailAddress, phoneNumber,
 dateOfBirth, specialty, yearsOfExperience)
VALUES
('111111111', 'Dr. John Doe', '3755 Chemin de la Cote
Sainte-Catherine',
 'j_doe@mcgill.ca', '514-340-8222',
 '1980-05-15', 'Cardiac Surgery', 20);
```

```sql
-- PATIENT
INSERT INTO Patient
(patientSIN, name, medicareCardNumber, insurance, policyNumber,
 homeAddress, emailAddress, phoneNumber, dateOfBirth,
 primaryDoctorSIN)
VALUES
('222222222', 'Joshua Visario', 'QC1234567890',
 'Beneva', 'POL1001',
 '1455 Blvd. De Maisonneuve Ouest', 'j_visar@live.concordia.ca',
 '514-714-0905', '1999-05-09', '111111111');


-- PHARMACEUTICAL COMPANY
INSERT INTO PharmaceuticalCompany
(pharmCompanyName, websiteAddress, emailAddress,
 companyAddress, phoneNumber)
VALUES
('Johnson & Johnson', 'https://johnson&johnson.com',
 'johnson&johnson.ca', '1 Kenvue Way, Summit, NJ 07901, United States',
 '514-999-9999');


-- PHARMACY
-- Main pharmacist will be assigned after Pharmacist is inserted.
INSERT INTO Pharmacy
(pharmacyID, pharmacyName, pharmAddress, websiteAddress,
 emailAddress, phoneNumber, mainPharmacistSIN)
VALUES
('PH0000001', 'Pharmaprix', '1500 Rue Sainte-Catherine Ouest,
Montreal',
 'https://pharmaprix.com',
 'pharmaprix@gmail.com', '514-933-4744', NULL);


-- PHARMACIST
INSERT INTO Pharmacist
(pharmacistSIN, name, homeAddress, emailAddress,
 phoneNumber, dateOfBirth, yearsOfExperience, pharmacyID)
VALUES
('333333333', 'Rebecca Do', '3755 Chemin de la Cote Sainte-Catherine,
Montreal',
```

```
 'rebecca.do@gmail.com', '514-555-0105',
 '2000-06-20', 2, 'PH0000001');


-- Assign the main pharmacist.
UPDATE Pharmacy
SET mainPharmacistSIN = '333333333'
WHERE pharmacyID = 'PH0000001';


-- Restore both mandatory NOT NULL and workplace-matching FK.
ALTER TABLE Pharmacy
MODIFY COLUMN mainPharmacistSIN CHAR(9) NOT NULL;

ALTER TABLE Pharmacy
ADD CONSTRAINT fk_pharmacy_main_pharmacist
    FOREIGN KEY (mainPharmacistSIN, pharmacyID)
    REFERENCES Pharmacist(pharmacistSIN, pharmacyID);

-- DRUG
INSERT INTO Drug
(tradeName, pharmCompanyName, formula)
VALUES
('Tylenol', 'Johnson & Johnson', 'C8H9NO2');


-- PRESCRIPTION
INSERT INTO Prescription
(prescriptionID, prescriptionDate, quantity,
 tradeName, pharmCompanyName, patientSIN, doctorSIN)
VALUES
(1, '2026-10-10', 30,
 'Tylenol', 'Johnson & Johnson', '222222222', '111111111');


-- CONTRACT
INSERT INTO Contract
(contractID, startDate, endDate, contractText,
 pharmCompanyName, pharmacyID, supervisorSIN)
VALUES
(1, '2026-01-01', '2027-12-31',
 'Supply agreement for Tylenol.',
 'Johnson & Johnson', 'PH0000001', '333333333');
```

```sql
-- SELLS
INSERT INTO Sells
(pharmacyID, tradeName, pharmCompanyName, price)
VALUES
('PH0000001', 'Tylenol', 'Johnson & Johnson', 19.99);


-- PRESCRIPTION PURCHASE
INSERT INTO PrescriptionPurchase
(purchaseID, prescriptionID, pharmacyID, approvedBySIN)
VALUES
(1, 1, 'PH0000001', '333333333');


-- Testings
SELECT * FROM Doctor;
SELECT * FROM Patient;
SELECT * FROM PharmaceuticalCompany;
SELECT * FROM Pharmacy;
SELECT * FROM Pharmacist;
SELECT * FROM Drug;
SELECT * FROM Prescription;
SELECT * FROM Contract;
SELECT * FROM Sells;
SELECT * FROM PrescriptionPurchase;

-- Checking the constraints
SHOW CREATE TABLE Pharmacy;
SHOW CREATE TABLE Pharmacist;
SHOW CREATE TABLE Prescription;
SHOW CREATE TABLE Sells;
```

c) [5 Points] Provide SQL statements that delete all data that you inserted in part (b).

```sql
-- Part C Delete records inserted in Part
II-----------------------------------------

-- Delete prescription purchase
```

```sql
DELETE FROM PrescriptionPurchase
WHERE purchaseID = 1;

-- Delete contract
DELETE FROM Contract
WHERE contractID = 1;

-- Delete Sells relationship
DELETE FROM Sells
WHERE pharmacyID = 'PH0000001'
  AND tradeName = 'Tylenol'
  AND pharmCompanyName = 'Johnson & Johnson';

-- Delete prescription
DELETE FROM Prescription
WHERE prescriptionID = 1;

-- Delete drug
DELETE FROM Drug
WHERE tradeName = 'Tylenol'
  AND pharmCompanyName = 'Johnson & Johnson';

-- Delete patient
DELETE FROM Patient
WHERE patientSIN = '222222222';

-- Temporarily remove circular FK
ALTER TABLE Pharmacy
DROP FOREIGN KEY fk_pharmacy_main_pharmacist;

-- Delete pharmacist
DELETE FROM Pharmacist
WHERE pharmacistSIN = '333333333';

-- Delete pharmacy
DELETE FROM Pharmacy
WHERE pharmacyID = 'PH0000001';

-- Restore the foreign key
ALTER TABLE Pharmacy
ADD CONSTRAINT fk_pharmacy_main_pharmacist
FOREIGN KEY (mainPharmacistSIN, pharmacyID)
REFERENCES Pharmacist(pharmacistSIN, pharmacyID);
```

```sql
-- Delete doctor
DELETE FROM Doctor
WHERE doctorSIN = '111111111';

-- Delete pharmaceutical company
DELETE FROM PharmaceuticalCompany
WHERE pharmCompanyName = 'Johnson & Johnson';


-- Testings

SELECT * FROM Doctor;
SELECT * FROM Patient;
SELECT * FROM Pharmacy;
SELECT * FROM Pharmacist;
SELECT * FROM Drug;
SELECT * FROM Prescription;
SELECT * FROM Contract;
SELECT * FROM PrescriptionPurchase;
SELECT * FROM Sells;
SELECT * FROM PharmaceuticalCompany;

SHOW TABLES;
```

d) [5 Points] Provide several SQL statements that delete all tables that you created in part (a).

```sql
-- Part d Delete Tables that you created in part a (Using
DROP)------------------------------

-- Remove circular foreign key constraint
ALTER TABLE Pharmacy
DROP FOREIGN KEY fk_pharmacy_main_pharmacist;

-- Drop dependent tables first
DROP TABLE IF EXISTS PrescriptionPurchase;
DROP TABLE IF EXISTS Contract;
DROP TABLE IF EXISTS Sells;
```

```
DROP TABLE IF EXISTS Prescription;

-- Drop remaining tables
DROP TABLE IF EXISTS Drug;
DROP TABLE IF EXISTS Patient;
DROP TABLE IF EXISTS Pharmacist;
DROP TABLE IF EXISTS Pharmacy;
DROP TABLE IF EXISTS PharmaceuticalCompany;
DROP TABLE IF EXISTS Doctor;

-- Verify that all tables have been deleted
SHOW TABLES;
```

Part III (25 points):

Express the following queries in **SQL:**

a) List the name of patients who are patients of at least three different doctors.

b) For every doctor who is also a patient, give the doctor's name and the name of the doctor treating him/her.

c) For every trade name drug that is not being sold by any pharmacy, provide the trade name of the drug and the name of the pharmaceutical company of the drug.

d) Provide details of all pharmaceutical companies that have contracts with at least 50 pharmacies. Details include name, website address, email address, address, phone number and the total number of contracts of the pharmaceutical company. Results should be displayed sorted in descending order by number of contracts.

e) List the name of all patients who do not have insurance. Details include SIN, name, Medicare card number, addresses, email address, phone number and date of birth. Results should be displayed sorted in ascending order by name.

```
-- Part 3 SQL Queries


-- a) List the name of patients who are patients of at least three
different doctors

SELECT P.name
FROM Patient P
JOIN Prescription R
    ON P.patientSIN = R.patientSIN
```

```sql
GROUP BY P.patientSIN, P.name
HAVING COUNT(DISTINCT R.doctorSIN) >= 3;


-- b) For every doctor who is also a patient, give the doctor's name
and the name of the doctor treating him/her.

SELECT D1.name AS DoctorPatient, D2.name AS TreatingDoctor
FROM Doctor D1
JOIN Patient P
    ON D1.doctorSIN  = P.patientSIN
JOIN Doctor D2
    ON P.primaryDoctorSIN = D2.doctorSIN;


-- c) For every trade name drug that is not being sold by any pharmacy,
provide the trade
-- name of the drug and the name of the pharmaceutical company of the
drug.

SELECT D.tradeName, D.pharmCompanyName
FROM Drug D
LEFT JOIN Sells S
    ON D.tradename = S.tradeName
    AND D.pharmCompanyName = S.pharmCompanyName
WHERE S.pharmacyID IS NULL;


-- d) Provide details of all pharmaceutical companies that have
contracts with at least 50
-- pharmacies. Details include name, website address, email address,
address, phone
-- number and the total number of contracts of the pharmaceutical
company. Results should
-- be displayed sorted in descending order by number of contracts.

SELECT
    PC.pharmCompanyName,
    PC.websiteAddress,
    PC.emailAddress,
    PC.companyAddress,
    PC.phoneNumber,
    COUNT(*) AS totalContracts
```

```sql
FROM PharmaceuticalCompany PC
JOIN Contract C
    ON PC.pharmCompanyName = C.pharmCompanyName
GROUP BY
    PC.pharmCompanyName,
    PC.websiteAddress,
    PC.emailAddress,
    PC.companyAddress,
    PC.phoneNumber
HAVING COUNT(DISTINCT C.pharmacyID) >= 50
ORDER BY totalContracts DESC;


-- e) List the name of all patients who do not have insurance. Details include SIN, name, Medicare card number,
-- addresses, email address, phone number and date of birth. Results should be displayed sorted in ascending order by name.

SELECT
    P.patientSIN,
    P.name,
    P.medicareCardNumber,
    P.homeAddress,
    P.emailAddress,
    P.phoneNumber,
    P.dateOfBirth
FROM Patient P
WHERE P.insurance IS NULL
ORDER BY P.name ASC;
```